

บทที่ 6

การออกแบบตัวแอปพลิเคชันฝั่งไคลเอนต์

6.1 รูปแบบโดยทั่วไปของฝั่งไคลเอนต์

การออกแบบฝั่งไคลเอนต์นี้จะมีอะไรที่พิเศษกว่าทางเซิร์ฟเวอร์ คือจะมีฟังก์ชันที่เพิ่มขึ้นมาเพื่อใช้สำหรับการวาดซึ่งการวาดนี้จะเป็นการวาด Map ในตำแหน่งที่ต้องการซึ่งได้อธิบายไว้ข้างแล้วในบทที่ 5 ในหัวข้อ 5.2

6.2 การทำงานฝั่งไคลเอนต์และฟังก์ชันต่างๆ

การทำงานในฝั่งไคลเอนต์ได้มีการทำงานหลายอย่างซึ่งเป็นการทำงานที่ต้องส่งข้อมูลไปที่เซิร์ฟเวอร์เพื่อให้เซิร์ฟเวอร์ไปดำเนินการต่างๆ แล้วก็ต้องทำการรอรับข้อมูลจากเซิร์ฟเวอร์เพื่อมาทำการเปลี่ยนแปลงค่าต่างๆ โดยในตัวไคลเอนต์เองจะมีหลายการทำงานแบ่งออกเป็น

6.2.1 การลงทะเบียน

เมื่อเริ่มเล่นฝั่งไคลเอนต์จะยังไม่มีค่า ID ซึ่งทำให้ไม่สามารถเริ่มเล่นได้ ทำให้ทางไคลเอนต์ต้องทำการลงทะเบียนก่อน เมื่อเลือกการลงทะเบียน ก็จะเข้าสู่หน้าต่างให้ป้อน IP และ Port ลงไปที่ TextField แล้วกดส่ง เมื่อกดส่งแล้วตัวไคลเอนต์จะทำการส่งค่า IP + Port ไปที่เซิร์ฟเวอร์

เมื่อกดส่งแล้วไคลเอนต์จะทำการเปิดการรับข้อมูล โดยเปิดการรับข้อมูลตามที่ได้ทำการป้อนใน TextField เพื่อที่จะรอรับแพ็กเกจตอบกลับจากเซิร์ฟเวอร์ได้

เมื่อไคลเอนต์ได้รับแพ็กเกจมาจากเซิร์ฟเวอร์ซึ่งเซิร์ฟเวอร์จะส่งค่าของ ID มาให้ ตัวไคลเอนต์ต้องทำการเก็บค่าเหล่านี้ไว้ในตัวแปรโดยข้อมูลที่ต้องบันทึกก็คือค่า IP Port และ ID

6.2.2 การเริ่มต้นของไคลเอนต์

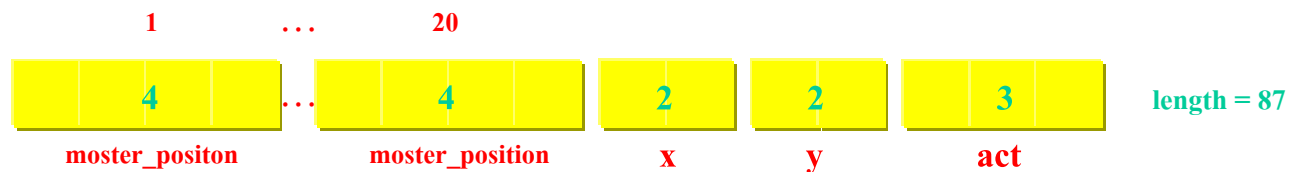
การเริ่มต้นของไคลเอนต์จะไม่สามารถเริ่มต้นได้ถ้ายังไม่มีค่า ID (ยังไม่ได้ทำการลงทะเบียน) ถ้าทำการลงทะเบียนแล้วจะสามารถเริ่มเล่นได้ โดยไคลเอนต์จะทำการดึงค่า IP และ Port มาทำการเปิดรับข้อมูล แล้วก็ดึงค่าของ ID มาเก็บไว้ในตัวแปร idreal

เมื่อเก็บค่าแล้วก็จะเป็นการส่งข้อมูลไปหาเซิร์ฟเวอร์เพื่อบอกให้เซิร์ฟเวอร์รู้ว่าไคลเอนต์ได้ทำการเข้ามาเริ่มเล่นแล้ว

เมื่อได้รับแพ็กเกจจากเซิร์ฟเวอร์ก็จะทำการเก็บค่าที่รับเข้ามาไว้ โดยแพ็กเกจที่รับเข้ามามีความยาวเท่ากับ 87 แล้วทำการเก็บค่าเหล่านี้ไว้ใน pos_x[] pos_y[] และ actorX และ actorY

ในการเก็บค่าของ actorX actorY นั้นทำโดยการตัดเอาค่า 4 ตัวสุดท้ายของแพ็กเกจที่รับเข้ามาแล้วทำการแบ่งเป็นอย่างละสอง สองตัวแรกเก็บไว้ที่ actorX ส่วนสองตัวหลังเก็บไว้ที่ actorY เสร็จแล้วก็ทำการตัดค่าแพ็กเกจที่รับเข้ามาให้เหลือแค่ 80 ตัว

ในการเก็บค่าของตัวศัตรูนั้นจะใช้ keep_mon()



รูปที่ 6.1 แพ็กเกจในกรณีเริ่มต้นของไคลเอนต์ที่ได้รับ

monster_position หมายถึง ค่าตำแหน่งของศัตรู

x หมายถึง ค่าตำแหน่ง x ของผู้เล่นที่เซิร์ฟเวอร์เป็นผู้กำหนดให้

y หมายถึง ค่าตำแหน่ง y ของผู้เล่นที่เซิร์ฟเวอร์เป็นผู้กำหนดให้

act หมายถึง ค่าการกระทำของตัวผู้เล่นที่เซิร์ฟเวอร์เป็นผู้กำหนดให้

keep_mon() เป็นฟังก์ชันที่มีไว้เพื่อทำการเก็บค่าตำแหน่งของศัตรูเก็บไว้ในตัวแปรอาร์เรย์ คือเก็บไว้ใน pos_x[] คือตำแหน่ง x ของศัตรู และ pos_y[] คือตำแหน่ง y ของศัตรู โดยในการเก็บค่าเหล่านี้ จะนำเอาค่าสตริงที่ได้จากแพ็กเกจที่ถูกตัดมาเหลือ 80 ตำแหน่งมาทำการตัด คือ ใน 2 ตำแหน่งแรกของสตริงก็จะให้เก็บไว้ใน pos_x[0] และ 2 ตำแหน่งถัดไปก็จะให้เก็บไว้ใน pos_y[0] และ 2 ตำแหน่งถัดไปก็จะเก็บไว้ใน pos_x[1] และ 2 ตำแหน่งถัดไปก็จะให้เก็บไว้ใน pos_y[1] ทำเช่นนี้ไปเรื่อยๆ จนถึงสิ้นสุดของสตริงก็จะได้ตำแหน่งของศัตรูทั้งหมด 20 ตัว เก็บไว้ในอาร์เรย์ของ int

เมื่อเก็บค่าต่างๆเสร็จสิ้นแล้วก็จะเป็นการเข้าสู่การวาดรูปที่หน้าจอเพื่อแสดงตำแหน่งของผู้เล่นด้วย

```
ColorTestCanvas a = new RagNo1.ColorTestCanvas();
```

```
a.repaint();
```

โดยที่ ColorTestCanvas เป็นคลาสที่ถูกสร้างขึ้นเพื่อใช้สำหรับวาดรูป ส่วน RagNo1 เป็นชื่อของคลาสหลักของแอปพลิเคชันของไคลเอนต์

6.2.3 การเดินและการหันของไคลเอนต์

เมื่อไคลเอนต์ทำการกดปุ่มทิศทางซึ่งหมายถึงการเดินหรือการหันหน้า แล้วจะเข้าสู่ส่วนของการตรวจสอบโดยแบ่งการตรวจสอบออกเป็น 4 แบบ

6.2.3.1 เมื่อกดลงก็จะทำการตรวจสอบว่าในตำแหน่งนั้นหันหน้าไปทางข้างหน้าหรือไม่ ถ้าไม่ได้หันหน้าไปทางข้างหน้าก็จะให้ ค่าการกระทำเป็นหันไปข้างหน้า แต่ถ้าหันไปทางข้างหน้าอยู่แล้วก็จะทำการตรวจสอบตำแหน่งที่อยู่ข้างล่างว่าเป็นหลุมหรือไม่ ถ้าเป็นหลุมแสดงว่าสามารถเดินได้ ก็จะให้เลื่อนตำแหน่งของผู้เล่นลงไปข้างล่างแล้วก็ทำการวาดรูปใหม่ แล้วส่งค่าไปให้กับเซิร์ฟเวอร์

ตรวจสอบว่าหันหน้าไปทางข้างหน้าหรือไม่ด้วย

```
if ((Map[actorX][actorY] == 'a') || (Map[actorX][actorY] == 'b'))
```

ให้ค่าการกระทำเป็นหันไปข้างหน้าด้วย

```
act = 'a'
```

ตรวจสอบตำแหน่งที่อยู่ข้างล่างว่าเป็นหล้าหรือไม่ด้วย

```
if (Map[actorX][actorY+1]== 'o')
```

เลื่อนตำแหน่งของผู้เล่นลงไปข้างล่างด้วย

```
actorY = actorY + 1
```

6.2.3.2 เมื่อกดขึ้นก็จะทำการตรวจสอบว่าในตำแหน่งนั้นหันหลังหรือไม่ ถ้าไม่ได้หันหลังก็จะให้ ค่าการกระทำเป็นหันไปข้างหลัง แต่ถ้าหันไปทางข้างหลังอยู่แล้วก็จะทำการตรวจสอบตำแหน่งที่อยู่ข้างบนว่าเป็นหล้าหรือไม่ ถ้าเป็นหล้าแสดงว่าสามารถเดินได้ ก็จะให้เลื่อนตำแหน่งของผู้เล่นขึ้นไปข้างบนแล้วก็ทำการวาดรูปใหม่ แล้วส่งค่าไปให้กับเซิร์ฟเวอร์

ตรวจสอบว่าหันหน้าไปทางข้างหลังหรือไม่ด้วย

```
if ((Map[actorX][actorY] == 'd')||(Map[actorX][actorY] == 'e'))
```

ให้ค่าการกระทำเป็นหันไปข้างหลังด้วย

```
act = 'd'
```

ตรวจสอบตำแหน่งที่อยู่ข้างบนว่าเป็นหล้าหรือไม่ด้วย

```
if (Map[actorX][actorY-1]== 'o')
```

เลื่อนตำแหน่งของผู้เล่นขึ้นไปข้างบนด้วย

```
actorY = actorY - 1
```

6.2.3.3 เมื่อกดซ้ายก็จะทำการตรวจสอบว่าในตำแหน่งนั้นหันซ้ายหรือไม่ ถ้าไม่ได้หันซ้ายก็จะให้ ค่าการกระทำเป็นหันไปทางซ้าย แต่ถ้าหันไปทางข้างซ้ายอยู่แล้วก็จะทำการตรวจสอบตำแหน่งที่อยู่ข้างซ้ายว่าเป็นหล้าหรือไม่ ถ้าเป็นหล้าแสดงว่าสามารถเดินได้ ก็จะให้เลื่อนตำแหน่งของผู้เล่นไปข้างซ้ายแล้วก็ทำการวาดรูปใหม่ แล้วส่งค่าไปให้กับเซิร์ฟเวอร์

ตรวจสอบว่าหันหน้าไปทางข้างซ้ายหรือไม่ด้วย

```
if ((Map[actorX][actorY] == 'j')||(Map[actorX][actorY] == 'k'))
```

ให้ค่าการกระทำเป็นหันไปข้างซ้ายด้วย

```
act = 'j'
```

ตรวจสอบตำแหน่งที่อยู่ข้างซ้ายว่าเป็นหล้าหรือไม่ด้วย

```
if (Map[actorX-1][actorY]== 'o')
```

เลื่อนตำแหน่งของผู้เล่นไปข้างซ้ายด้วย

```
actorX = actorX - 1
```

6.2.3.4 เมื่อกดขวาก็จะทำการตรวจสอบว่าในตำแหน่งนั้นหันขวาหรือไม่ ถ้าไม่ได้หันขวาก็จะให้ ค่าการกระทำเป็นหันไปทางขวา แต่ถ้าหันไปทางข้างขวาอยู่แล้วก็จะทำการตรวจสอบตำแหน่งที่อยู่ข้างขวว่าเป็นหล้าหรือไม่ ถ้าเป็นหล้าแสดงว่าสามารถเดินได้ ก็จะให้เลื่อนตำแหน่งของผู้เล่นไปข้างขวาแล้วก็ทำการวาดรูปใหม่ แล้วส่งค่าไปให้กับเซิร์ฟเวอร์

ตรวจสอบว่าหันหน้าไปทางข้างซ้ายหรือไม่ด้วย

```
if ((Map[actorX][actorY] == 'g')||(Map[actorX][actorY] == 'h'))
```

ให้ค่าการกระทำเป็นหันไปข้างซ้ายด้วย

```
act = 'g'
```

ตรวจสอบตำแหน่งที่อยู่ข้างซ้ายว่าเป็นหลัหรือไม่ว

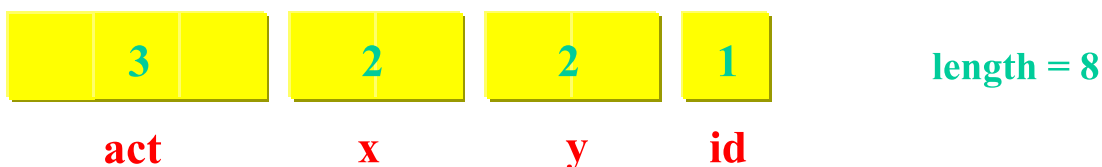
```
if (Map[actorX+1][actorY]== 'o')
```

เลื่อนตำแหน่งของผู้เล่นไปข้างซ้ายด้วย

```
actorX = actorX + 1
```

ในการวาดรูปใหม่จะใช้ repaint()

และในการส่งไปที่เซิร์ฟเวอร์จะใช้ send() โดยจะมีค่า sendoption = 0



รูปที่ 6.2 แพ็กเกจในการเดินและการหันของไคลเอนที่จะส่ง

act หมายถึง ค่าที่แสดงการกระทำของตัวผู้เล่น

x หมายถึง ค่าตำแหน่ง x ของตัวผู้เล่น

y หมายถึง ค่าตำแหน่ง y ของตัวผู้เล่น

id หมายถึง ค่า id ของตัวผู้เล่น

เมื่อได้รับแพ็กเกจที่มีความยาวเท่ากับ 8 แสดงว่าได้รับแพ็กเกจเกี่ยวกับการเดินหรือการหันก็จะทำการเข้าฟังก์ชัน StringToInt() และก็เข้าฟังก์ชัน keepoint()

StringToInt() เป็นฟังก์ชันที่เปลี่ยนค่าที่รับเข้ามาให้เป็น int โดยจะทำการเก็บค่า ID ที่ได้รับเข้ามาไว้ในตัวแปร id

keepoint() เป็นฟังก์ชันที่ใช้สำหรับเก็บตำแหน่งของศัตรูซึ่งจะเก็บไว้ใน point[id]

6.2.4 การฆ่าศัตรู

เมื่อทำการกดปุ่มเพื่อฆ่าศัตรู (ปุ่มฟันซึ่งเป็นปุ่มเดียวกันกับปุ่มสู้กับผู้เล่นคนอื่น) แล้วก็จะดูว่าผู้เล่นหันหน้าไปทางไหน ถ้าหันไปทางไหนก็จะให้เปลี่ยนเป็นการกระทำหันหน้าไปทางนั้นแล้วกำลังฟัน เมื่อเปลี่ยนค่าแล้วก็จะทำการวาดรูป เมื่อวาดแล้วก็ตรวจสอบดูว่าในตำแหน่งที่มันต้องการฟันนั้นมีตัวศัตรูอยู่หรือไม่ ถ้ามีก็จะทำการให้ค่า sendoption เป็น 1 เมื่อเสร็จแล้วก็จะทำการเปลี่ยนการกระทำเป็นอย่างเดิมก่อนทำการฟัน เสร็จแล้วก็จะทำการส่งค่าไปให้กับเซิร์ฟเวอร์ โดยจะแบ่งได้เป็น 4 แบบ

6.2.4.1 หันหน้าไปข้างหน้า

ตรวจสอบและให้ค่าการกระทำเป็นฟันด้วย

```
if ((Map[actorX][actorY] == 'a') || (Map[actorX][actorY] == 'b'))
```

act = 'c'

ตรวจสอบว่ามีศัตรูอยู่หรือไม่ด้วย

if(Map[actorX][actorY+1] == 'z')

6.2.4.2 หันหน้าไปข้างหลัง

ตรวจสอบและให้ค่าการกระทำเป็นฟันด้วย

if((Map[actorX][actorY] == 'd')||(Map[actorX][actorY] == 'e'))

act = 'f'

ตรวจสอบว่ามีศัตรูอยู่หรือไม่ด้วย

f(Map[actorX][actorY-1] == 'z')

6.2.4.3 หันหน้าไปข้างซ้าย

ตรวจสอบและให้ค่าการกระทำเป็นฟันด้วย

if((Map[actorX][actorY] == 'j')||(Map[actorX][actorY] == 'k'))

act = 'l'

ตรวจสอบว่ามีศัตรูอยู่หรือไม่ด้วย

if(Map[actorX-1][actorY] == 'z')

6.2.4.4 หันหน้าไปข้างขวา

ตรวจสอบและให้ค่าการกระทำเป็นฟันด้วย

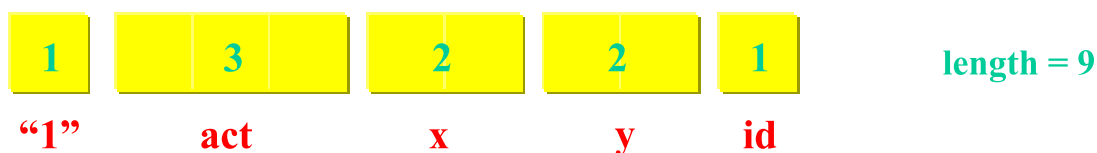
if((Map[actorX][actorY] == 'g')||(Map[actorX][actorY] == 'h'))

act = 'i'

ตรวจสอบว่ามีศัตรูอยู่หรือไม่ด้วย

if(Map[actorX+1][actorY] == 'z')

เมื่อเสร็จแล้วก็จะทำการส่งด้วย send() โดยจะมีค่า sendoption = 1



รูปที่ 6.3 แพ็กเกจในกรณีการฆ่าศัตรูของไคลเอนท์ที่จะส่ง

“1” หมายถึง ใส่เพื่อแบ่งให้รู้ว่าเป็นแพ็กเกจของการฆ่าศัตรู

act หมายถึง ค่าที่แสดงการกระทำของผู้เล่น
 x หมายถึง ค่าตำแหน่ง x ของตัวผู้เล่น
 y หมายถึง ค่าตำแหน่ง y ของตัวผู้เล่น
 id หมายถึง ค่า id ของตัวผู้เล่น

เมื่อได้รับแพ็กเกจที่มีความยาวเท่ากับ 12 ถ้าใช่แสดงว่าเป็นแพ็กเกจเกี่ยวกับการฆ่าศัตรู
 ก็จะทำการเข้าฟังก์ชัน Killed_mon()

Killed_mon() จะเป็นฟังก์ชันที่ทำการตรวจสอบดูว่าแพ็กเกจที่รับเข้ามามีค่า ID ตรงกับ
 ค่า id ของตนเองหรือไม่ ถ้าเท่ากันแสดงว่าเราได้ฆ่าศัตรูแล้ว ก็จะทำการเพิ่มคะแนนให้กับตัวเอง แล้ว
 ทำการเอาค่าที่รับมานั้นซึ่งมีค่าตำแหน่งของศัตรูตัวใหม่อยู่ด้วยก็จะทำการเอาค่าตำแหน่งของศัตรูตัวใหม่
 มาทำการแทนที่ของศัตรูตัวเก่าด้วย แล้วยังทำการเก็บค่า Item ที่เกิดขึ้นด้วย โดยเมื่อเข้าฟังก์ชันนี้แล้วจะ
 ทำการแปลงค่าที่รับเข้ามาเป็นค่าต่างๆ

ทำการเปรียบเทียบ ID ที่รับมากับ ID ของตนเองด้วย

```
if (id_kill.compareTo(idreal) == 0)
```

ทำการเก็บค่าตำแหน่งของศัตรูตัวใหม่ด้วย

```
pos_x[id_mon_kill] = new_mon_x
```

```
pos_y[id_mon_kill] = new_mon_y
```

เก็บค่า Item ด้วย

```
item[it] = item_xy
```

6.2.5 การเก็บ Item

ในการเก็บ Item นั้นจะเป็นเหมือนการเดินแต่เพียงจะทำการตรวจสอบว่าใน
 ตำแหน่งที่จะเดินไปนั้นเป็นตำแหน่งของ Item หรือไม่ ถ้าเป็นก็จะทำการกำหนด sendoption ให้เท่ากับ 2
 แล้วทำการส่ง

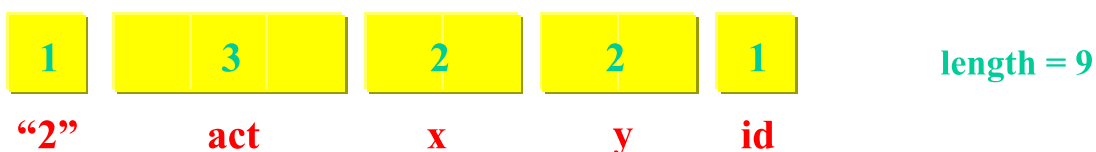
ตรวจสอบว่าในตำแหน่งที่จะเดินไปเป็น Item ด้วย

```
if (Map[actorX][actorY+1] == 't') หรือ
```

```
if (Map[actorX][actorY-1] == 't') หรือ
```

```
if (Map[actorX-1][actorY] == 't') หรือ
```

```
if (Map[actorX+1][actorY] == 't')
```



รูปที่ 6.4 แพ็กเกจในกรณีการเก็บ Item ของโคลอนที่จะส่ง

“2” หมายถึง ใส่เพื่อแบ่งให้รู้ว่าเป็นแพ็กเกจของการเก็บ Item

act หมายถึง ค่าที่แสดงการกระทำของตัวผู้เล่น

x หมายถึง ค่าตำแหน่ง x ของตัวผู้เล่น

y หมายถึง ค่าตำแหน่ง y ของตัวผู้เล่น

id หมายถึง ค่า id ของตัวผู้เล่น

เมื่อได้รับแพ็กเกจที่มีความยาวเท่ากับ 7 ถ้าใช่แสดงว่าเป็นแพ็กเกจเกี่ยวกับการเก็บ Item ก็จะทำการเข้าฟังก์ชัน Get_item()

Get_item() จะเป็นฟังก์ชันที่ทำการตรวจสอบว่าแพ็กเกจที่รับเข้ามามีค่า ID ตรงกับค่า id ของตนเองหรือไม่ ถ้าเท่ากันแสดงว่าเราได้เก็บ Item แล้ว ก็จะทำการเพิ่มความสามารถให้เราตามแต่ Item ที่เราเก็บได้ เสร็จแล้วก็จะทำการเปลี่ยนค่าใน item[] ให้มีค่าเป็น 0

ทำการเปรียบเทียบ ID ที่รับมากับ ID ของตนเองด้วย

```
if (id_keep.compareTo(idreal) == 0)
```

ทำการเปลี่ยนค่าใน item[] ให้เท่ากับ 0 ด้วย

```
item[id_item] = 0
```

6.2.6 การสู้กับผู้เล่นคนอื่น

เมื่อทำการกดปุ่มเพื่อสู้กับผู้เล่นคนอื่น (ปุ่มฟันซึ่งเป็นปุ่มเดียวกันกับปุ่มฆ่าศัตรู) แล้วก็ถือว่าผู้เล่นหันหน้าไปทางไหน ถ้าหันไปทางไหนก็จะให้เปลี่ยนเป็นการกระทำหันหน้าไปทางนั้นแล้ว กำลังฟัน เมื่อเปลี่ยนค่าแล้วก็จะทำการวาดรูป เมื่อวาดแล้วก็ตรวจสอบดูว่าในตำแหน่งที่มันต้องการฟัน นั้นมีผู้เล่นคนอื่นอยู่หรือไม่ ถ้ามีก็จะทำการให้ค่า sendoption เป็น 3 เมื่อเสร็จแล้วก็จะทำการเปลี่ยนการกระทำเป็นอย่างเดิมก่อนทำการฟัน เสร็จแล้วก็จะทำการส่งค่าไปให้กับเซิร์ฟเวอร์

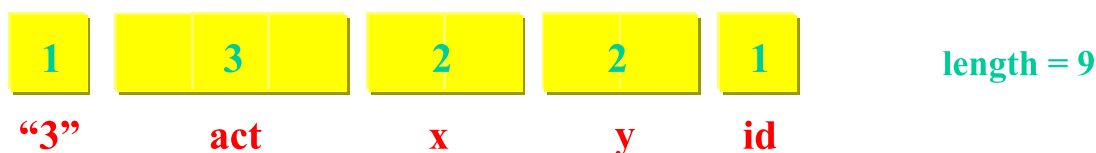
ตรวจสอบว่าในตำแหน่งที่ต้องการฟันเป็นผู้เล่นคนอื่นหรือไม่ด้วย

```
if ((Map[actorX][actorY+1] == 'a') || (Map[actorX][actorY+1] == 'b'))
```

```
|| (Map[actorX][actorY+1] == 'd') || (Map[actorX][actorY+1] == 'e'))
```

```
|| (Map[actorX][actorY+1] == 'g') || (Map[actorX][actorY+1] == 'h'))
```

```
|| (Map[actorX][actorY+1] == 'j') || (Map[actorX][actorY+1] == 'k'))
```



รูปที่ 6.5 แพ็กเกจในกรณีการสู้กับผู้เล่นคนอื่นของไคลเอนที่จะส่ง

“3” หมายถึง ใส่เพื่อแบ่งให้รู้ว่าเป็นแพ็กเกจของการสู้กับผู้เล่นคนอื่น

act หมายถึง ค่าที่แสดงการกระทำของตัวผู้เล่น

x หมายถึง ค่าตำแหน่ง x ของตัวผู้เล่น

y หมายถึง ค่าตำแหน่ง y ของตัวผู้เล่น

id หมายถึง ค่า id ของตัวผู้เล่น

เมื่อได้รับแพ็กเกจที่มีความยาวเท่ากับ 6 ถ้าใช่แสดงว่าเป็นแพ็กเกจเกี่ยวกับการฆ่าศัตรูก็จะทำการเข้าฟังก์ชัน Killed_actor()

Killed_actor() จะเป็นฟังก์ชันที่ทำการตรวจสอบดูว่าแพ็กเกจที่รับเข้ามาซึ่งจะมีค่า ID อยู่ 2 ค่าโดยจะเป็น ID ของผู้ฟันกับ ID ของผู้ถูกฟัน โดย ID ของตนเองตรงกับ ID ของผู้ฟันก็จะไปเพิ่มคะแนนให้กับตัวเอง แต่ถ้า ID ของตนเองตรงกับ ID ของผู้ถูกฟันก็จะไปลดพลังชีวิตของตนเอง

ตรวจสอบว่า ID ของตนเองตรงกับ ID ของผู้ฟัน แล้วทำการเพิ่มคะแนนด้วย

```
if (id_kill.compareTo(idreal) == 0)
```

```
    score = score + 1
```

ตรวจสอบว่า ID ของตนเองตรงกับ ID ของผู้ถูกฟัน แล้วทำการลดพลังชีวิตด้วย

```
if (id_killed.compareTo(idreal) == 0)
```

```
    blood = blood - 2
```

6.2.7 การออกจากโปรแกรม

เมื่อไคลเอนต์ต้องการออกจากโปรแกรม ก็จะกดปุ่ม * เพื่อให้เลือกออกชั้น เมื่อกดแล้วตัวไคลเอนต์ก็จะแสดงหน้าต่างเพื่อให้เลือกออกชั้น เมื่อเลือก Exit Game แล้วก็ไคลเอนต์ก็จะทำการเก็บค่าคะแนนปัจจุบันที่อยู่ในตัวแปร score แล้วก็ค่า ID ของตัวเองเพื่อส่งไปให้กับเซิร์ฟเวอร์

6.2.8 การวาดรูป

การวาดรูปจะเริ่มด้วยการกำหนดค่าต่างให้ว่าให้เป็นรูปอะไรเช่นถ้าเป็น 'm' ก็ให้แทนด้วยรูปกำแพง ถ้าเป็น 'a' ก็ให้แทนด้วยรูปผู้เล่นหันหน้าไปทางข้างหน้า แล้วก็เข้า createmap()

createmap() เป็นฟังก์ชันที่ใช้สำหรับการกำหนดค่า Map ว่าในแต่ละตำแหน่งเป็นอะไรโดยมีการกำหนดค่าของตัวผู้เล่นอยู่ในนี้ด้วย Map[actorX][actorY] = act

ต่อไปก็จะเป็นการตรวจสอบว่ามีศัตรูตัวอื่นหรือไม่ ถ้ามีก็จะทำการเปลี่ยนค่าใน Map ตรวจสอบด้วย

```
if(point[j] != null)
```

ทำการเปลี่ยนค่าใน Map ด้วย

```
Map[receiveactorX][receiveactorY] = ค่าที่เก็บไว้ใน point[ ]
```

ต่อไปก็จะเป็นการตรวจสอบค่าของศัตรูที่เก็บตำแหน่งไว้ใน pos_x[] และ pos_y[]

```
ทำโดย Map[pos_x[k]][pos_y[k]] = 'z';
```

ต่อไปก็จะเป็นการตรวจสอบค่าของ Item ที่เก็บไว้ใน item[]

ทำโดย `if(item[k] != 0)`

`Map[item[k]/100][item[k]%100] = 't';`

เป็นการวาดรูปต่างๆ ที่อยู่ใน Map โดยจะทำการวาดเพียงตำแหน่งใน Map ที่ต้องการ โดยตำแหน่งที่ต้องการนั้นจะรู้ได้จากการทำให้ตัวผู้เล่นต้องอยู่ตรงกลางของจอ แล้วก็จะทำการดูว่าในตำแหน่งที่ต้องการวาดเป็นรูปอะไรก็ให้วาดรูปนั้นออกไป